

Istruzioni. Durante lo svolgimento del test, attenersi rigorosamente alle seguenti istruzioni.

- Gli esercizi devono essere consegnati attraverso il sito Moodle, al seguente indirizzo:

<https://elearning.unica.it/course/view.php?id=188>

- Ogni esercizio deve essere consegnato su un file con estensione `.ml`. Non sono ammesse altre modalità di consegna.
- Non è possibile consegnare alcun esercizio dopo la scadenza stabilita. È possibile re-inviare ogni esercizio un numero arbitrario di volte entro la scadenza stabilita, senza subire penalizzazioni.
- Non è ammesso alcun tipo di collaborazione.
- Se viene richiesto che una funzione abbia un nome specifico, la funzione deve avere quel nome.
- Il file contenente la soluzione non deve contenere esempi (anche commentati).
- Nelle soluzioni degli esercizi 7 e 8 il tipo deve essere contenuto nel file.

1. Scrivere una funzione `foo` con il seguente tipo:

`foo: 'a -> ('a -> 'b) -> 'b`

2. Si consideri la successione $(a_n, b_n)_{n \in \mathbb{N}}$ di coppie di numeri interi data da:

$$\begin{aligned} a_0 &= 1 & b_0 &= 1 \\ a_{n+1} &= \begin{cases} a_n & \text{se } b_n \text{ dispari} \\ a_n \times b_n & \text{altrimenti} \end{cases} & b_{n+1} &= \begin{cases} a_n + b_n & \text{se } a_n \text{ dispari} \\ a_n \times b_n & \text{altrimenti} \end{cases} \end{aligned}$$

Scrivere una funzione `seq: int -> (int * int)` tale che l'espressione `seq n` valuta alla coppia (a_n, b_n) . Nota: è consentito definire funzioni ausiliarie.

3. Date due funzioni $f, g: \mathbb{Z} \rightarrow \mathbb{Z}$, si definisca la funzione $f \circ g: \mathbb{Z} \rightarrow \mathbb{Z}$ come segue:

$$(f \circ g)(x) = \begin{cases} |g(x)| & \text{se } g(x) > f(x) \\ |f(x)| & \text{se } f(x) > g(x) \\ 10 + f(x) & \text{altrimenti} \end{cases}$$

Definire una funzione `comp` con il seguente tipo:

`comp: (int -> int) -> (int -> int) -> (int -> int)`

e tale che $\text{comp } f \ g = f \circ g$.

4. Scrivere una funzione `foo` con il seguente tipo:

```
foo: 'a list -> 'b list -> 'b -> 'a
```

5. Definire una funzione `fool` con tipo:

```
'a list -> int -> ('a -> 'a) -> 'a list
```

in modo che `fool l n f` sia la lista ottenuta da `l` mappando gli ultimi `n` elementi con la funzione `f`, e lasciando inalterati i rimanenti. Ad esempio,

```
fool [1;2;3;4;5;6;7;8;9;10] 3 (fun x -> x*2) = [1;2;3;4;5;6;7;16;18;20]
```

6. Scrivere una funzione `foo` con il seguente tipo:

```
foo: 'a list -> 'b list -> ('b -> 'a) -> 'a list
```

7. Si consideri il seguente tipo:

```
type 'a tree = Empty | Node of 'a * 'a tree * 'a tree
```

Definire una funzione `sumEven` con tipo:

```
'int tree -> int
```

in modo che `sumEven t` restituisca la somma dei valori dei nodi di profondità pari di `t` (si assume che la radice sia al livello 0). Ad esempio:

```
# let t1 = Node (1, Node(2, Node( 3, Empty, Empty), Node(4, Empty, Empty)),
Node ( 5, Empty, Empty));;
```

```
# sumEven t1;;
```

```
- : int = 8
```

8. Si consideri la seguente definizione di tipo:

```
type exp = Pippo
```

```
| Conc of exp * exp
```

```
| Revert of exp
```

```
| Invappend of exp * exp;;
```

Definire una funzione **eval** che assegni ad un espressione di tipo **exp** un valore di tipo **int list**, interpretando **Pippo** con la lista `[0;1]`, **Conc of exp * exp** con la concatenazione del valore della prima espressione con quello della seconda, **Revert of exp** con la lista ottenuta invertendo il valore dell'espressione di tipo **exp**, **Invappend of exp * exp** con la lista ottenuta concatenando il valore della seconda espressione al valore della prima espressione.